Test Name: JAVA DSA Practice 1

Test Code: SNPU00003

Description:

Welcome to the placement test. This test consists of 1 section: Coding Challenge. Please read the instructions for each section carefully before starting.

Section Summary

Section	No. of Questions	Marks (+ / -)
Coding Challenge	2	-

Coding Challenge Section

Duration: 30 minutes

Questions: 2

1

Problem Statement: You are part of a software development team working on a smart grid system that monitors electricity consumption in a residential neighborhood. Each household reports hourly energy consumption values in kilowatt-hours (kWh) over a day. These consumption values are collected in an array where each element represents the energy consumed during a specific hour.

Your task is to write a program that calculates the sum of even energy consumption values recorded during the day. This sum helps the energy management system identify patterns where energy usage is balanced or aligned with grid supply during specific hours, aiding in optimizing energy distribution and reducing wastage.

Constraints:

$1 \leq arr.size \leq 10^5$

$1 \leq arr[i] \leq 10^5$

Example 1:

Input:

n= 5

arr[] = [2, 4, 3, 5, 6]

Output:12

Explanation: 2,4,6 are even and their sum is 12

Example 2:

Input:

n= 3

arr[] = [1, 1, 1]

Output: 0

Test Cases:

Input: 5 8 5 6 1 2 — VISIBLE

Expected Output: 16

Input: 6 9 6 8 3 4 1 — VISIBLE

Expected Output: 18

Input: 5 1 2 7 6 4 — HIDDEN

Expected Output: 12

Input: 4 2 8 10 7 — HIDDEN

Expected Output: 20

Input: 8 2 3 7 8 19 1 5 10 — HIDDEN

Expected Output: 20

Input: 3 6 8 10 — HIDDEN

Expected Output: 24

Explanation:

Code in C++:

```
#include <iostream>
using namespace std;

// Function to calculate sum of even elements
int sumOfEvenElements(int arr[], int n) {
    int sum = 0;

    for (int i = 0; i < n; i++) {
        if (arr[i] % 2 == 0) {
            sum += arr[i];
        }
    }

    return sum;
}

int main() {
    int n;
    cin >> n;

    if (n <= 0) {
        cout << endl;
        return 0;
    }

    int arr[n];
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    int result = sumOfEvenElements(arr, n);
    cout  << result << endl;

    return 0;
}
```


Marks: 25 points

2

Problem Statement: The City Heritage Museum is upgrading its digital security system to enhance visitor tracking and improve crowd-flow management. As part of this upgrade, every visitor entering the museum is required to wear an identification badge that contains a unique single-character code. When visitors pass through designated checkpoints, their badge codes are automatically scanned and recorded in a central monitoring system. Throughout the day, thousands of visitors enter and move through the museum, resulting in a long sequence of recorded badge codes.

Museum security analysts want to study these sequences of badge scans to learn more about visitor patterns. One specific metric they are interested in is identifying the longest time interval during which every visitor who passed through a checkpoint had a distinct badge code, meaning no two consecutive badge scans during that period belonged to the same visitor or to two visitors with identical badge codes. This type of information helps the museum understand when visitor flow was most diverse, which can assist in staffing, exhibit planning, and emergency-response readiness.

The sequence of badge scans for a particular day is represented as a string s, where each character corresponds to the badge code of a visitor as they were scanned in order. A single badge code may appear multiple times in the overall sequence because visitors may pass multiple checkpoints or return to the same exhibit.

Your task is to analyze this string and determine the length of the longest continuous substring in which no badge code is repeated. In other words, you must find the size of the largest sequence of consecutive visitors where all badge codes are unique. This value represents the longest period of completely distinct visitor movement and is crucial for the museum's security and operations analysis.

Constraints :

- String contains lower case characters
- 1 <= string length <= 1000

Example 1:

Input: s = "abcabcbb"

Output: 3

Explanation: The longest sequence without repeating badges is "abc" with length 3. Other valid sequences like "bca" and "cab" also have length 3.

Example 2:

Input: s = "bbbbb"

Output: 1

Explanation: All visitors have the same badge code "b", so the longest unique sequence is just one visitor with length 1.

Example 3:

Input: s = "pwwkew"

Output: 3

Explanation: The longest sequence is "wke" with length 3. Note that "pwke" would not count as it's not a continuous sequence in the original order.

Test Cases:

Input: tmmzuxt

Expected Output: 5

VISIBLE

Input: tmmzuxt

Expected Output: 5

HIDDEN

Input: abcdef

Expected Output: 6

HIDDEN

Input: aabbcc

Expected Output: 2

HIDDEN

Input: nfpdmpi

Expected Output: 5

HIDDEN

Explanation:

Code in Python:

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        if not s:
            return 0

        char_index_map = {}
        max_length = 0
        left = 0

        for right, ch in enumerate(s):
            if ch in char_index_map:
                left = max(left, char_index_map[ch] + 1)

            char_index_map[ch] = right
            max_length = max(max_length, right - left + 1)

        return max_length

if __name__ == "__main__":
    s = input().strip()
    sol = Solution()
    result = sol.lengthOfLongestSubstring(s)
    print(result)
```

Marks: 25 points